

Computer Vision Practical Report:

CONVOLUTION KERNEL

1. AIM

To find the structural boundary of an image using a 3×3 high-pass convolution edge detection kernel (e.g., Ridge/Laplacian filter) with `cv2.filter2D()` in OpenCV, display the original and boundary images, and save the result.

2. PROCEDURE

- Load the image and convert it to grayscale using `cv2.cvtColor()`.
- Define a 3×3 boundary detection convolution kernel (e.g., center coefficient $= 8$ and all surround coefficients $= -1$).
- Convolve the image with the custom kernel using `cv2.filter2D()` to highlight boundary transitions.
- Normalize or take the absolute values of the response using `cv2.convertScaleAbs()` to clip pixel values into valid 8-bit ranges $[0, 255]$.
- Display the original image and boundary-extracted output using Matplotlib, and save the result using `cv2.imwrite()`.

3. PROGRAM CODE

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load image
img_bgr = cv2.imread('E28-IMG.jpg')
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# 2. Define Boundary Extraction Kernel (3x3 Laplacian filter)
kernel = np.array([
    [-1, -1, -1],
```

```

    [-1, 8, -1],
    [-1, -1, -1]
], dtype=np.float32)

# 3. Apply 2D convolution to find boundary
boundary = cv2.filter2D(gray, -1, kernel)

# 4. Display on a single page
fig, axes = plt.subplots(1, 2, figsize=(8, 4))

axes[0].imshow(img_rgb)
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(boundary, cmap='gray')
axes[1].set_title('Extracted Boundary')
axes[1].axis('off')

plt.tight_layout()
plt.show()

```

4. OUTPUT



5. RESULT

The Python script executed successfully. The input image was loaded.

